

routes\main_routes.py

```
from flask import Blueprint, render_template, request, redirect, url_for
from datetime import datetime, timezone, timedelta
import math
from services.orchestrator import data_orchestrator
from utils.filters import FilterService

main_bp = Blueprint('main', __name__)
filter_service = FilterService()

@main_bp.route("/")
def index():
    """Main dashboard route"""
    try:
        print("[Flask] Loading dashboard...")

        year = request.args.get('year', type=int)
        month = request.args.get('month', type=int)
        day = request.args.get('day', type=int)
        severity_filter = request.args.get('severity')
        search_query = request.args.get('q')
        timeline_years = request.args.get('timeline_years', default=1, type=int)

        if timeline_years not in [1, 2, 3, 5]:
            timeline_years = 1

        print(f"[Flask] Filters: year={year}, month={month}, day={day},
severity={severity_filter}, search={search_query}, timeline_years={timeline_years}")

        dashboard_data = data_orchestrator.get_dashboard_data(
            year=year,
            month=month,
            day=day,
            severity_filter=severity_filter,
            timeline_years=timeline_years,
            load_historical=True
        )

        if search_query:
            filtered_cves = filter_service.filter_by_search(dashboard_data['latest_cves'],
search_query)

        from collections import Counter
        severity_counts = Counter()
        for cve in filtered_cves:
            severity = cve.get('Severity', 'UNKNOWN').upper()
            if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
                severity_counts[severity] += 1
            else:
                severity_counts['UNKNOWN'] += 1

        dashboard_data['metrics'] = {
            'CRITICAL': severity_counts.get('CRITICAL', 0),
            'HIGH': severity_counts.get('HIGH', 0),
            'MEDIUM': severity_counts.get('MEDIUM', 0),
            'LOW': severity_counts.get('LOW', 0),
            'UNKNOWN': severity_counts.get('UNKNOWN', 0)
        }
        dashboard_data['total_cves'] = len(filtered_cves)
        dashboard_data['latest_cves'] = filtered_cves

        total = len(filtered_cves)
        if total > 0:
            dashboard_data['severity_percentage'] = {}
            for key in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']:
                percentage = round((severity_counts.get(key, 0) * 100 / total))
                dashboard_data['severity_percentage'][key] = f"{percentage}%"
            else:
                dashboard_data['severity_percentage'] = {k: "0%" for k in ['CRITICAL', 'HIGH', 'MEDIUM',
'LOW', 'UNKNOWN']}

        dashboard_data['note_text'] = f"Showing search results for '{search_query}'"
```

```

return render_template(
    "pages/dashboard.html",
    metrics=dashboard_data['metrics'],
    total_cves=dashboard_data['total_cves'],
    available_years=dashboard_data['available_years'],
    available_months=dashboard_data['available_months'],
    timeline_data_days=dashboard_data['timeline_data_days'],
    timeline_data_years=dashboard_data['timeline_data_years'],
    severity_stats=dashboard_data['metrics'],
    severity_percentage=dashboard_data['severity_percentage'],
    cwe_radar=dashboard_data['cwe_radar'],
    cwe_radar_all=dashboard_data['cwe_radar_all'],
    cwe_radar_weighted=dashboard_data['cwe_radar_weighted'],
    cwe_radar_descriptions=dashboard_data['cwe_radar_descriptions'],
    historical_loaded=dashboard_data.get('historical_loaded', False),
    year_filter=year,
    month_filter=month,
    day_filter=day,
    severity_filter=severity_filter,
    search_query=search_query,
    timeline_years=timeline_years
)

except Exception as e:
    print(f"[Flask] Dashboard error: {e}")
    import traceback
    traceback.print_exc()

return render_template(
    "pages/dashboard.html",
    metrics={'CRITICAL': 0, 'HIGH': 0, 'MEDIUM': 0, 'LOW': 0, 'UNKNOWN': 0},
    total_cves=0,
    available_years=list(range(datetime.now().year, 1998, -1)),
    available_months=list(range(1, 13)),
    timeline_data_days={'labels': [], 'values': []},
    timeline_data_years={'labels': [], 'values': []},
    severity_stats={'CRITICAL': 0, 'HIGH': 0, 'MEDIUM': 0, 'LOW': 0, 'UNKNOWN': 0},
    severity_percentage={'CRITICAL': '0%', 'HIGH': '0%', 'MEDIUM': '0%', 'LOW': '0%',
    'UNKNOWN': '0%'},
    cwe_radar={'indices': [], 'labels': [], 'values': []},
    cwe_radar_all={},
    cwe_radar_weighted={'indices': [], 'labels': [], 'values': []},
    cwe_radar_descriptions={},
    historical_loaded=False,
    year_filter=None,
    month_filter=None,
    day_filter=None,
    severity_filter=None,
    search_query=None,
    timeline_years=1
)

@main_bp.route("/vulnerabilities")
def vulnerabilities():
    """Vulnerabilities listing page"""
    try:
        year = request.args.get('year', type=int)
        month = request.args.get('month', type=int)
        day = request.args.get('day', type=int)
        severity_filter = request.args.get('severity')
        search_query = request.args.get('q')
        page = request.args.get('page', default=1, type=int)
        per_page = 20

        print(f"[Flask] Vulnerabilities filters: year={year}, month={month}, day={day},
        severity={severity_filter}, search={search_query}")

        all_cves = data_orchestrator.get_filtered_cves(year=year, month=month, day=day)
        print(f"[Flask] Got {len(all_cves)} CVEs from orchestrator")

        if severity_filter:
            all_cves = filter_service.filter_by_severity(all_cves, severity_filter)
            print(f"[Flask] After severity filter: {len(all_cves)} CVEs")

        if search_query:
            all_cves = filter_service.filter_by_search(all_cves, search_query)

```

```

print(f"[Flask] After search filter: {len(all_cves)} CVEs")

all_cves = sorted(all_cves, key=lambda x: x.get('Published', ''), reverse=True)

total_results = len(all_cves)
total_pages = max(1, math.ceil(total_results / per_page))
current_page = max(1, min(page, total_pages))
start_index = (current_page - 1) * per_page
end_index = start_index + per_page
cves_page = all_cves[start_index:end_index]

page_numbers = filter_service.generate_page_numbers(current_page, total_pages)

note_text = filter_service.generate_note_text(year, month, day)
if severity_filter:
    note_text += f" (filtered by {severity_filter.lower()} severity)"
if search_query:
    note_text += f" (search: '{search_query}')"

print(f"[Flask] Returning {len(cves_page)} CVEs for page {current_page}")

return render_template(
    "pages/vulnerabilities.html",
    latest_cves=cves_page,
    year_filter=year,
    month_filter=month,
    day_filter=day,
    severity_filter=severity_filter,
    search_query=search_query,
    available_years=list(range(datetime.now().year, 1998, -1)),
    available_months=list(range(1, 13)),
    current_page=current_page,
    total_pages=total_pages,
    page_numbers=page_numbers,
    total_results=total_results,
    note_text=note_text
)

except Exception as e:
    print(f"[Flask] Vulnerabilities page error: {e}")
    import traceback
    traceback.print_exc()

    return render_template(
        "pages/vulnerabilities.html",
        latest_cves=[],
        total_results=0,
        note_text="Error loading vulnerabilities"
    )

@main_bp.route("/cve/<cve_id>")
def cve_detail(cve_id):
    """CVE detail page"""
    try:
        cve = data_orchestrator.get_cve_detail(cve_id)
        return render_template("pages/cve_detail.html", cve=cve)
    except Exception as e:
        print(f"[Flask] CVE detail error: {e}")
        return render_template("pages/cve_detail.html", cve={'ID': cve_id, 'Description': 'Error loading CVE details'})

```